



Evaluation and comparison of conventional Clock Tree Synthesis (CTS) with Multi-Source Clock Tree Synthesis (MSCTS)

Sreeram Unny, Kim Wee Ng, Keng Kuay Koh

Infineon Technologies
Singapore

www.infineon.com

ABSTRACT

As the semiconductor industry progressively moves towards high performance ASIC design, it becomes very critical for design engineers to construct a zero-skew robust-by-construction clock network or utilize useful skew for meeting the aggressive performance target. This paper evaluates and compares various clock methodologies such as conventional clock tree synthesis, clock tree synthesis with Concurrent Clock and Data Optimization (CCD) and Multi-Source Clock Tree Synthesis (MSCTS): Regular MSCTS and Structural MSCTS. A detailed description of the MSCTS flow and the associated challenges are provided with aim to help designers make the right choices prior to implementing MSCTS on their designs. From the results obtained during the evaluation process, various engineering trade-offs could be observed and are highlighted and explained. The possible impact of MSCTS on other aspects of the design flow is also briefly summarized. The evaluation is performed using Synopsys IC Compiler 2 (ICC 2) on a 40nm design block operating at 300 MHz.

Table of Contents

1. Introduction	4
1.1 Motivation:.....	4
1.2 Background:	4
2. MSCTS Flow and Challenges.....	8
2.1 SMSCTS.....	8
2.1.1 Clock Mesh Creation:	9
2.1.2 Mesh Driver Insertion:	9
2.1.3 GCT Synthesis:	10
2.1.4 Mesh Modelling:.....	11
2.1.5 LST Synthesis	11
2.1.6 Clock Mesh Routing:.....	12
2.1.7 Clock Mesh Analysis:.....	12
2.1.8 Synthesis and Optimization of Clocks:	13
2.2 RMSCTS	13
2.2.1 Tap Assignment:	14
3. Impact of MSCTS on PnR Flow	14
3.1 Design (RTL)	14
3.2 Synthesis.....	14
3.3 DFT	14
3.4 Floorplan and Placement.....	15
3.5 IR Drop	15
3.6 STA	15
4. Results.....	15
4.1 Key Performance Indices.....	15
4.2 Summary.....	17
5. Future Work.....	18
6. Conclusions	18
7. Acknowledgements.....	18
8. References	18

Table of Figures

Figure 1. Conventional Clock Tree.....	5
Figure 2. CTS Flow in ICC2.....	5

Figure 3. CCD and CTS	6
Figure 4. SMSCTS Clock Structure	7
Figure 5. RMSCTS Clock Structure.....	8
Figure 6. SMSCTS Flow in ICC2	8
Figure 7. GCT Synthesis.....	11
Figure 8. Clock Mesh Routing	12
Figure 9. RMSCTS Flow in ICC2	13

Table of Tables

Table 1. Performance Comparisons between Clock Methodologies	16
--	----

1. Introduction

1.1 Motivation:

In an era where process nodes are scaling down rapidly and the demand for high performance Application-Specific Integrated Circuits (ASIC) is increasing exponentially, it is expected that the overall design complexity of an ASIC to be on the rise. Amidst a vast plethora of design elements and parameters that needs to be considered for a successful implementation of high performance ASICs, nothing demands more attention than the design of a variation-insensitive, zero-skew and power efficient clock network. Since majority of the contemporary digital circuits employ synchronous clocking for their operation, the clock network has a significant impact on not just the functionality but also on Power, Performance and Area (PPA).

The primary motivation behind this evaluation was to check the feasibility of implementing MSCTS on an industrial design and observe the associated flow-specific challenges and its impact on other phases of the design flow. Once the feasibility check was complete and the flow was successfully executed, the emphasis was placed on performance improvement by skew minimisation. The evaluation and comparison is performed on a 40nm design block operating at 300MHz.

This paper details and compares the various methodologies currently used for building clock networks. The prime focus is placed on Multi-Source Clock Tree Synthesis (MSCTS). Apart from delineating the MSCTS flow and the associated challenges, the impact of this flow on other aspects of the ASIC design flow is also briefly summarised.

The following Synopsys tools were used for MSCTS evaluation:

- **IC Compiler 2 (ICC2):** ICC2 was the primary tool used for the implementation of the design block. A dedicated PnR database was created for the evaluation.
- **HSPICE:** Owing to the inability of ICC2/PrimeTime to perform timing analysis on a multi-driver driven net, HSPICE was used for simulating the design (Transistor-Level). The simulation results were then back-annotated to ICC2 for obtaining the timing QoR.
- **StarRC:** StarRC was used for the parasitic extraction of the mesh net. The resulting SPEF was then back-annotated to ICC2.

Please note that the terms 'strap', 'grid' and 'mesh' are used interchangeably in this paper.

1.2 Background:

The fundamental goal of any clock distribution architecture is to deliver the clock signal from its source (clock definition point) to the sinks (flops/macros/latches) without inducing significant clock skew. Clock skew can be categorized as following:

- **Global Clock Skew:** The maximum skew between sinks in a specific clock domain that do not have a data path between them.
- **Local Clock Skew:** The maximum skew between sinks having a data path between them.

From the definitions, it can be deduced that the local skew is more important for timing closure while the global skew is a measure of the overall quality of the clock network.

The following architectures are primarily used for designing a clock network:

- **Clock Tree:** The conventional approach for building clock distribution networks employs a 'tree' structure. This is shown in the following figure:

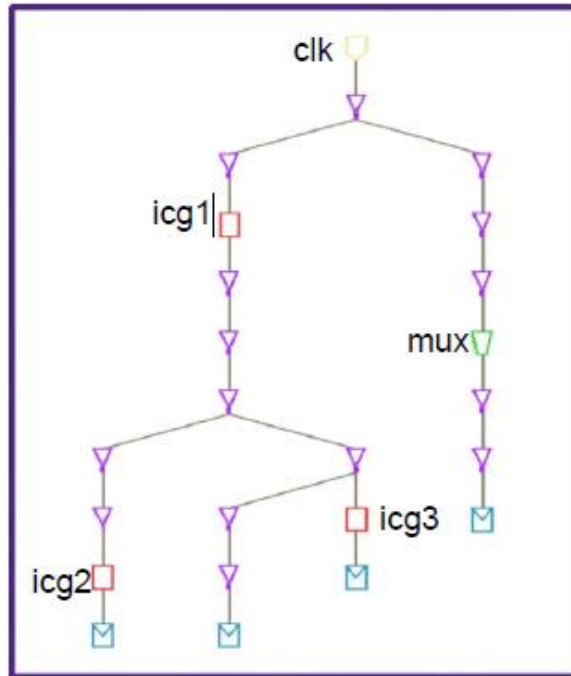


Figure 1. Conventional Clock Tree

As can be seen from **Figure 1**, the clock signal starts propagating from the clock root (clk) and traverses through many 'branches' before reaching the sinks (blue). The tool has inserted many CTS cells for balancing the clock tree. Since the signal propagates through a large number of unique paths, the clock network is highly susceptible to On-Chip Variations (OCV). Furthermore, performance gets compromised owing to the latency differences between these unique paths that could result in a higher skew. Despite these disadvantages, this approach is still quite ubiquitous in the industry owing to its high power efficiency, minimal resource usage and ease of implementation.

The following flow diagram depicts the CTS flow in ICC2:

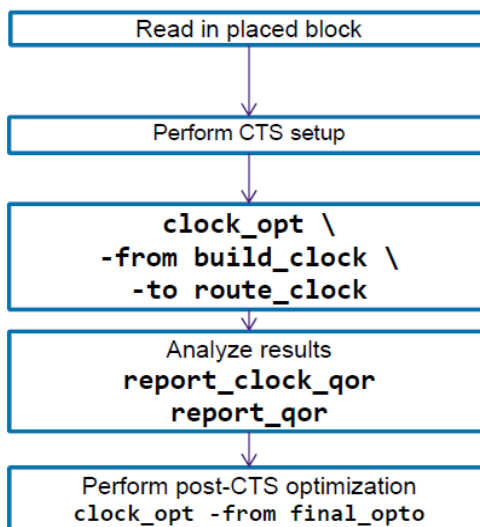


Figure 2. CTS Flow in ICC2

- Concurrent Clock and Data Optimization (CCD):** As the clock frequency began to rise steadily, designers were not able to achieve timing closure by using the conventional clock tree approach. To mitigate these problems, a new approach called CCD was used by the industry. CCD is a clock design methodology whereby skew is intentionally introduced to improve timing margins on certain critical timing paths. As the name implies, the improvements in timing is due to the concurrent optimization of both the clock and data paths. Optimizations include clock tree cell sizing, sizing of flops, re-parenting, buffer removal and buffering. Apart from a better timing QoR, the clock trees are also optimized for area/power. These area/power optimizations are performed with awareness of their impact on the data paths.

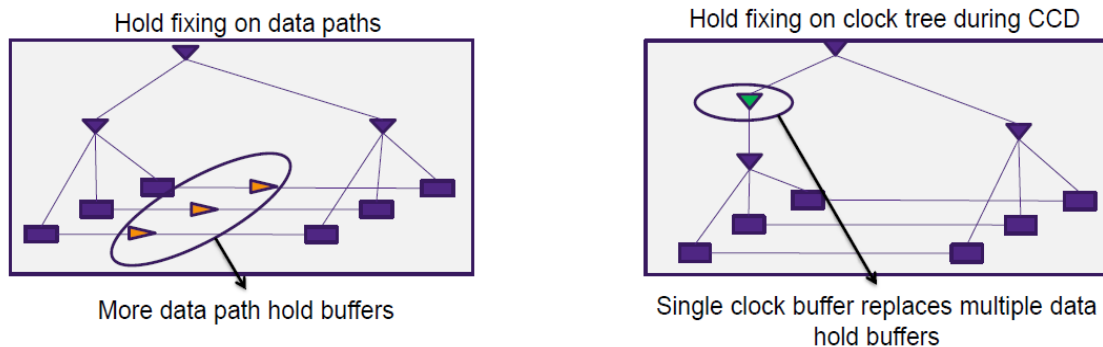


Figure 3. CCD and CTS

Figure 3 highlights the differences between CCD and CTS in fixing hold timing violations. While CTS fixes the violation by adding multiple hold buffers in the data path, CCD modifies the clock path for achieving the same.

CCD methodology is supported by Synopsys. In ICC2, CCD makes an impact on 'clock_opt' and 'route_opt'. CCD in clock_opt can be enabled by setting the application option 'clock_opt.flow.enable_ccd' to true prior to running 'clock_opt'. Similarly, CCD in 'route_opt' can be enabled using the same application option before running 'route_opt'. It should be noted that the tool does not resize flip-flops during conventional CTS while they get resized during CCD (if timing is improved).

- Multi-Source Clock Tree Synthesis (MSCTS):** MSCTS is a clock distribution architecture whereby a grid/mesh replaces the conventional tree topology. Typically used in high performance microprocessors and FPGAs, such a grid-based architecture has more tolerance to On-Chip Variation (OCV) and a better performance across corners than traditional clock tree structures.

Clock networks designed using MSCTS constitutes three distinct parts:

- Mesh:** A clock mesh is a uniform grid of metal straps laid out across the chip for distributing the clock signal to the sequential elements. While the redundant loops/paths in mesh are responsible for facilitating a high OCV tolerance, the mesh is also responsible for burning a lot of power due to the huge RC.
- Global Clock Tree (GCT):** While the mesh drives the clock sinks, the mesh itself is driven by a network of clock cells referred to as the Global Clock Tree (GCT).

Designing a low-skew GCT is a very important part of mesh-based clock architecture.

- c. **Local Sub-Trees (LST):** The section of the clock network after the mesh is collectively referred to as the LST. The exact structure of these LSTs is highly dependent on the flavour of the MSCTS clock network.

MSCTS comes in two different structural variants/flavours:

- a. **Structural MSCTS (SMSCTS):** SMSCTS refers to a variant of MSCTS, whereby all the sinks are directly driven by the mesh straps. In SMSCTS, the sequence of clock cells in the incoming netlist is not allowed to change. Thus, the tool is not allowed to insert any buffers maintaining the 'structure' of the clock network. Since (un)buffering is restricted, the tool can only optimize the network by merging, splitting and sizing the existing clock logic to meet the CTS goals. The term 'SMSCTS' in ICC2 terminology is equivalent to 'clock mesh' in ICC terminology. The following figure captures the general structure of a clock network implemented using SMSCT:

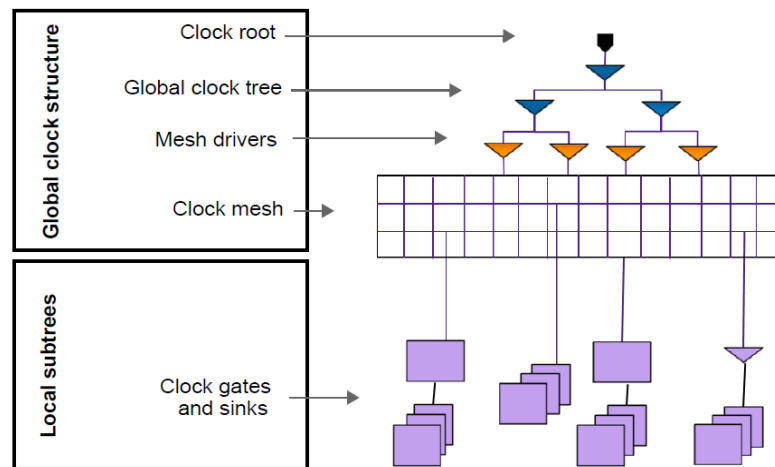


Figure 4. SMSCTS Clock Structure

- b. **Regular MSCTS (RMSCTS):** RMSCTS refers to a variant of MSCTS, whereby the sinks are driven by a network of drivers called 'tap' drivers. These drivers are driven directly by the clock mesh. In RMSCTS, the depth of the clock tree is modified during the implementation of the clock tree. As the name implies, this approach closely correlates with the regular way of implementing clock networks. The term 'RMSCTS' in ICC2 terminology is equivalent to 'MSCTS' in ICC terminology. The following figure captures the general structure of a clock network implemented using RMSCT:

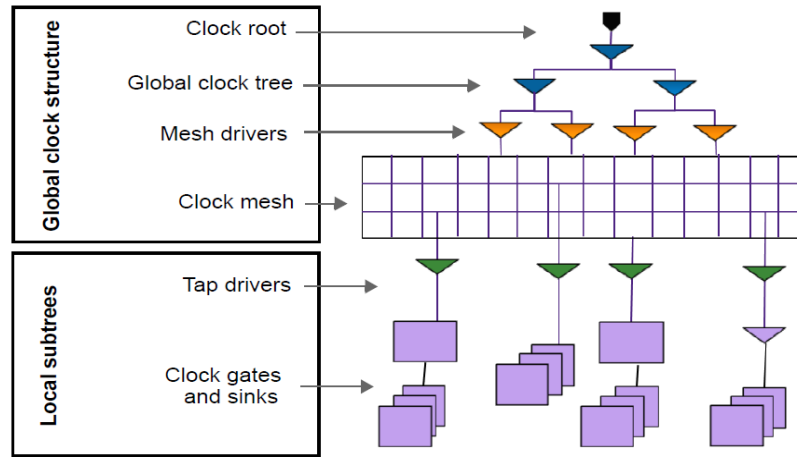


Figure 5. RMSCTS Clock Structure

2. MSCTS Flow and Challenges

As noted in the previous section, both RMSCTS and SMSCTS come with a GCT and a grid. It's only the LST that proves to be a differentiating factor among these two topologies. Thus, it seems only natural for these similarities to be reflected in the implementation flow. The following sections describe the RMSCTS/SMSCTS design parameters, implementation flow and the challenges that were encountered during each phase of the design. The resolution for these challenges is also described.

It is assumed that the design is placed and an optimized database is available for MSCTS.

2.1 SMSCTS

The following flowchart describes the SMSCTS flow:

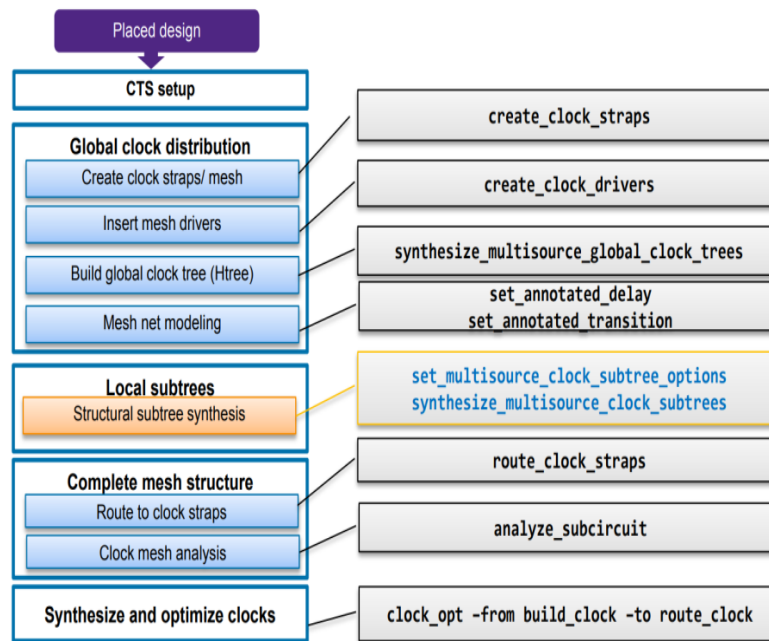


Figure 6. SMSCTS Flow in ICC2

Figure 6 lists the commands for executing each phase of the SMSCTS flow in ICC2. The following section describes each phase in detail along with the challenges they pose.

2.1.1 Clock Mesh Creation:

- Command
 - The clock straps are created using the command 'create_clock_straps'. ICC2 makes it possible to vary the pitch independently along the horizontal and vertical directions.
- Design Parameters and Observations

The pitch P is the most important design parameter associated with a clock grid since it decides the density of interconnections. The pitch has an impact on not just the latency ($P \propto \text{Latency}$) and slew ($P \propto \frac{1}{\text{Slew}}$) but also the power consumption of the clock network ($P \propto \text{Power}$). The aforementioned relation between slew and pitch assumes a constant number of mesh drivers.

 - Typically, the top layers of the metal stack are used for constructing the clock mesh owing to their lower resistance. However, the layers of the PG network should be accounted while constructing the clock straps since the interaction of the PG network with the clock network could result in SI issues.
 - The width W for the straps can be varied. As width increases, the resistance R drops ($W \propto \frac{1}{R}$) while the loading Cap for the mesh drivers increases ($W \propto Cap$).
 - The boundary of the mesh was a design variable and was chosen based on the placement and the distribution of the endpoints. Flexibility of the mesh boundary implies that the straps could be built in a rectilinear fashion.
- Challenges
 - The block shape itself was the biggest challenge. However, it was later observed that the mesh drivers could only be placed in a symmetric manner. Hence, the boundary of the block had to be modified to enable such a symmetric distribution.
 - The tool offers the flexibility to build the mesh on any clock net in the incoming netlist. Deciding the mesh net proved to be a challenge since there were no single nets with the desired fanout. The clock network in the incoming netlist had to be modified to obtain a high-fanout net (>10000). However, such a strategy would not work in a real scenario because the implementation team cannot modify the clock structure in the incoming netlist. Thus, the RTL designer has to make sure that the clock structure is as minimal as possible.

2.1.2 Mesh Driver Insertion:

- Command
 - The mesh drivers are inserted using the command 'create_clock_drivers'.

- Design Parameters and Observations

- The number of mesh drivers is a design parameter. This was decided based on the loading. The load to be driven by the mesh drivers is a function of the MSCTS topology:

In SMSCTS,

$$\text{Load Capacitance} = \text{Mesh Cap} + \text{Sink Pin Capacitance}$$

In RMSCTS,

$$\text{Load Capacitance} = \text{Mesh Cap} + \text{Tap Driver Pin Capacitance}$$

- Greater the number of mesh drivers for the same clock grid, the smaller the latency. However, area and power gets penalized. It is important to note that the driver count is proportional to the complexity of the GCT network driving these buffers.
- For a given pitch P of the mesh network, the slew is inversely proportional to the number of mesh drivers ($\text{Slew} \propto \frac{1}{\text{Mesh Driver Count}}$).
- Apart from the count, the drive strength of the mesh drivers is another design parameter. For a given number of mesh drivers and pitch P , the drive strength D is inversely proportional to the slew ($\text{Slew} \propto \frac{1}{D}$). However, higher drive strength results in an increased cell area. The drive strength will also impact the latency of the clock signal across the mesh. Even though multiple drivers are used in parallel to drive large loads, it is usually preferable to have higher strength drivers available for MSCTS.

- Challenges

- ICC2 places these drivers in a symmetric manner to facilitate the construction of an H-Tree during GCT Synthesis. The symmetry implies a rectangular placement of these drivers on the floorplan. Such a symmetrical arrangement becomes problematic (GCT Synthesis will fail) if the straps were to be built rectilinearly. Thus, the mesh boundary needs to be chosen such that the straps are rectangular too.

2.1.3 GCT Synthesis:

- Command

- The GCT is synthesized and detail routed using the command 'synthesize_multisource_global_clock_trees'.

- Design Parameters and Observations

- ICC2 enables the designer to select the cell types for synthesizing the global tree.
- The complexity of the GCT depends on the number of mesh drivers. The H-Tree allows for a near-zero skew at the inputs of the mesh drivers. Keeping this skew minimal is important not just for performance but also power consumption. Varied clock arrival times at the mesh-driver inputs result in short-circuit currents through the mesh and can burn power.
- GCT synthesis would fail if there is a large displacement in the location of the mesh drivers ($>0.3 \times \text{pitch}$).

Figure 7 shows the GCT after synthesis and detail routing:

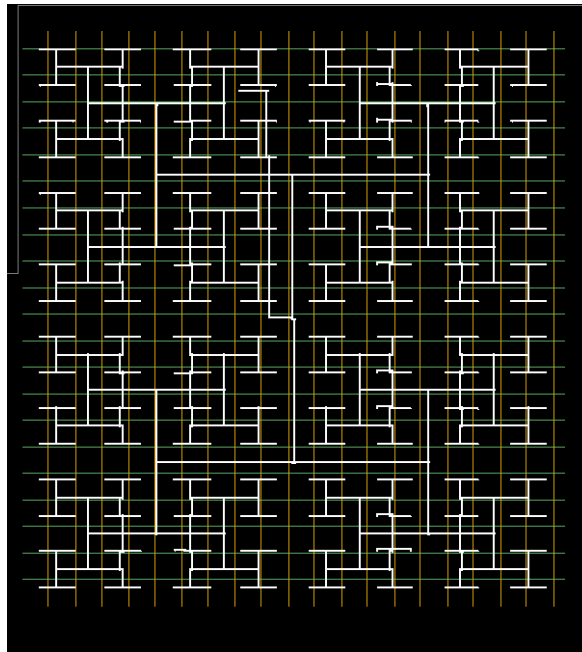


Figure 7. GCT Synthesis

2.1.4 Mesh Modelling:

- Command
 - The clock mesh is modelled using the commands 'set_annotated_delay', 'set_annotated_transition' and 'set_disable_timing'.
- Observations
 - The motivation behind modelling is to build and optimize the LST. The designer is required to manually annotate the cell/net delays and transitions so that ICC2 propagates the clock with realistic values. It should be noted that ICC2 cannot time a multi-driver driven net.
 - Since the clock logic is simple, this paper doesn't discuss the impact of modelling on the LST network.

2.1.5 LST Synthesis

- Command
 - LST synthesis is performed using the mega command 'synthesize_multisource_clock_subtrees'. Prior to the execution of 'synthesize_multisource_clock_subtrees', it is important to configure the synthesis and optimization using 'set_multisource_clock_subtree_options'.
- Design Parameters and Observations
 - The tool performs global routing of all clock nets during this phase and optimizes the LST by merging/splitting/sizing/relocating the clock logic.
 - Since no buffer insertion or removal is performed (except for level balancing), the number of levels to each sink remain the same before and after the subtree synthesis.
 - In later versions of ICC2, the designer is allowed to specify a target latency/level

count and the tool would honour these targets by buffering.

- It should be noted that LST optimization can result in a very large pin load for the mesh drivers. Hence, the incoming netlist should have a sufficient number of clock levels.

2.1.6 Clock Mesh Routing:

- Command
 - The mesh is detail routed to the rest of the clock network using 'route_clock_straps'.
- Design Parameters and Observations
 - The routes links the mesh straps to the mesh drivers and the 1st-level sinks of the mesh. These sinks could be tap-drivers (RMSCTS) or registers (SMSCTS without level balancing) or clock buffers (SMSCTS with level balancing).
 - The following figure illustrates the two routing topologies currently supported by ICC2: Fishbone and Comb.

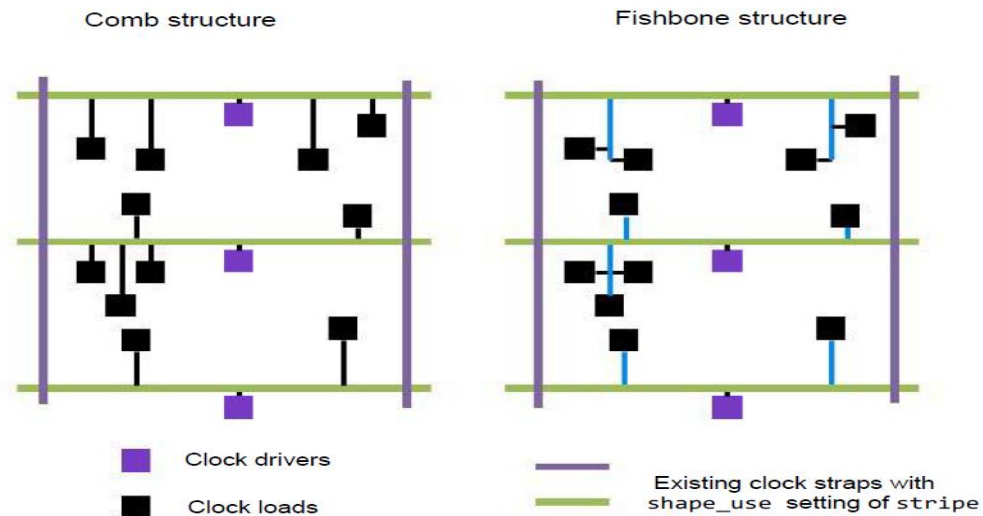


Figure 8. Clock Mesh Routing

- From **Figure 8**, it can be seen that the mesh drivers are always connected to the straps irrespective of the routing topologies. However, the connectivity of the sinks differs. In comb routing, each sink is individually connected to the nearest strap via a finger. In fishbone, multiple sinks share the same finger.
- This paper employs a comb topology for mesh routing.

2.1.7 Clock Mesh Analysis:

- Command
 - Clock mesh analysis is performed to obtain the final QoR (skew, latency, WNS, TNS etc.). The mega command for analyzing the mesh is 'analyze_subcircuit'.
- Design Parameters and Observations
 - Though HSPICE was used for this evaluation, ICC2 also supports FineSim and NanoSim.

- The command performs extraction (ICC2), tool invocation, netlist preparation, transistor-level simulation and back-annotation.
- The tool enables the user to utilize SPEF files generated by 3rd party vendors. However, there is a specific naming convention to be used in such cases.
- For a successful circuit simulation, two files are mandatory:
 - **SPICE Model Library:** Contains the device models at various corners. This file is typically provided by the library vendor.
 - **Driver Subcircuit Files:** Contains the SPICE model (circuit-level) for all the cells in the clock network (MUX, ICG, Buffers etc.). Accuracy of the simulation can be greatly enhanced if these models contain transistor-level parasitic information.

2.1.8 Synthesis and Optimization of Clocks:

- Once the delays are back-annotated successfully, the designer can proceed to completing the SMSCTS flow by using the command 'clock_opt'.
- During 'clock_opt', those sinks not covered by the mesh straps would be balanced with the SMSCTS sinks. Furthermore, the command will synthesize, route and optimize all other clocks in the design (if the design comprises of multiple clock domains).
- It is important to mark the mesh net as 'don't touch' before running 'clock_opt' so that it doesn't get modified during synthesis.

2.2 RMSCTS

The following flowchart describes the RMSCTS flow:

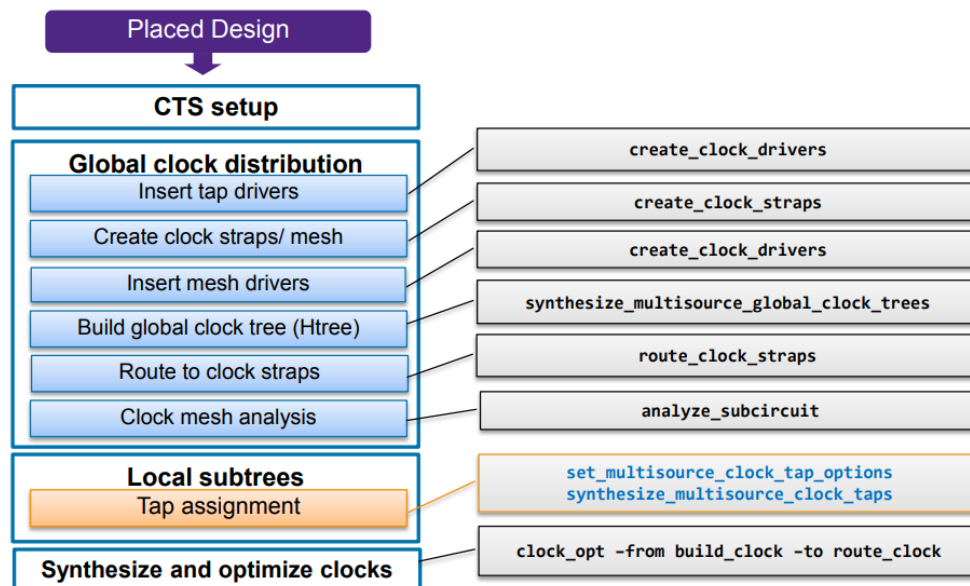


Figure 9. RMSCTS Flow in ICC2

Comparison of **Figure 6** and

Figure 9 illustrates a high degree of correlation when it comes to implementing SMSCTS and RMSCTS. Since a detailed explanation was provided for most of the steps in RMSCTS flow, the emphasis of this section is primarily on 'Tap Assignment'.

Since both tap-drivers and mesh-drivers are inserted using 'create_clock_drivers', no separate explanation is provided below for tap driver insertion.

2.2.1 Tap Assignment:

- Command
 - Tap assignment is carried out using 'synthesize_multisource_clock_taps'. Prior to the execution of 'synthesize_multisource_clock_taps', it is important to configure synthesis and optimization using 'set_multisource_clock_tap_options'.
- Design Parameters and Observations
 - During tap assignment, the sinks are clustered and assigned to a specific tap-driver. This process is fully automated.
 - Tap assignment is placement-driven.
 - The number of tap drivers is a design parameter that has an impact on the drivability of the mesh drivers. The number of sub-trees is equal to the number of tap drivers.
 - The tap drivers should be equivalent with respect to the loads and the clock.
 - The fanout cone for each tap driver may not be the same.

Once the tap-assignment is complete, the designer can proceed to completing the RMSCTS flow by using the command 'clock_opt'. The tool performs 'regular' CTS from each of these tap drivers to its assigned sinks.

Since the sub-trees in RMSCTS comprises of clock trees built during 'clock_opt', there is no requirement for modelling the mesh. The entire clock network is fully built and routed prior to the mesh analysis (unlike in SMSCTS).

3. Impact of MSCTS on PnR Flow

This section briefly describes the impact of MSCTS on other aspects of the PnR flow.

3.1 Design (RTL)

MSCTS affects the design of the clock network in the incoming netlist. Ideally, the number of levels in the clock network should be kept as minimal as possible. This enables tying all the endpoints to the clock straps and obtaining a perfectly balanced clock network. However, the depth of the clock network is usually constrained by functionality, application, power efficiency etc. A simple strategy would be to reuse as much clock logic as possible rather than adding redundant clock logic to the network. The designer can also use 'create_clock' after the logic blocks to obtain a clock tree whereby a single source is driving many sinks.

3.2 Synthesis

The designer should be cautious when performing power optimization (Power Compiler) during Synthesis due to the large number of ICGs that gets added to the netlist. Though they play a positive role in reducing the clock power, they tend to increase the depth of the clock network. As mentioned in the previous sections, greater the depth of the clock network in the incoming netlist, more the problems when executing MSCTS.

3.3 DFT

There are a few challenges posed by MSCTS for Design for Test (DFT):

- Since there is a grid, it is not possible to test the mesh drivers since they do not cause any change in the logic functionality. Thus, there is no observability.
- If a few drivers do fail by stuck-at-faults, this can result in a short-circuit current through the mesh. Such a cross-current would have an impact on the reliability of the mesh network due to the gradual degradation of the clock straps.
- One possible solution would be to devise a mechanism to disconnect the mesh straps from the faulty drivers and reconnect them to the good drivers.
- Further analysis is required in this direction since the impact of a few faulty mesh drivers on the clock and timing QoR is not well known.

3.4 Floorplan and Placement

The floorplan and the cell distribution play a dominant role in the MSCTS flow. This is due to the symmetric distribution of the mesh/tap drivers. The shape of the block should be decided such that there is a rectangular area for placing the clock sinks. Additionally, care must be taken so that the cells are placed near to the mesh straps (placement blockages). It is important to note that the cell placement impacts the overall boundary of the clock mesh.

3.5 IR Drop

Another area of interest is IR Drop. The decrement in skew tends to worsen the IR drop due to the simultaneous switching of all the sequential elements. From this outlook, RMSCTS seems to have an edge over SMSCTS due to the structure of the sub-trees. Factors such as high-drive mesh-drivers, RC in the mesh straps and a minimal number of ICGs contribute to high power consumption in MSCTS-based designs.

3.6 STA

We have seen how the reliance on multiple buffers for driving a single net presents some challenges for delay calculation. Apart from the availability of an accurate SPICE model, performing accurate STA poses few challenges:

- The waveform used for the simulation is an ideal waveform. The tool generates this waveform on the basis of the clock frequency. As known, using an ideal waveform can induce inaccuracies in delay calculation. Fortunately, it is possible to transform this ideal waveform into a realistic one by shifting the simulation clock source a few stages before the required node.
- RC extraction is important for successfully back-annotating the delays after simulation. It was observed during the evaluation that the fanout of the clock mesh has a detrimental effect on the extraction process. Thus, in SMSCTS where we have a huge number of sinks being driven by the mesh, extraction using ICC2 gets really problematic. However, the issue was resolved by interfacing StarRC with ICC2.
- The accuracy of the timing at the top-level should be analyzed in great detail if some hierarchical blocks utilize MSCTS while others employ the conventional CTS.

4. Results

4.1 Key Performance Indices

The following performance indices were used for the evaluation and comparison of MSCTS against CTS (without CCD) and CTS (with CCD):

- Clock Latency
- Local Skew
- Global Skew
- Buffer Count
- Clock Depth (Levels)
- Worst Negative Slack (WNS)
- Total Negative Slack (TNS)

The clock mesh was implemented for a single clock domain in the block. The clock was chosen based on a few factors like clock frequency, criticality of the timing paths, number of sinks etc. The rest of the clocks were constructed using the conventional approach.

It is important to note that CTS/CCD and MSCTS (RMSCTS/SMSCTS) were executed on two different design databases (say DB1 and DB2 respectively). This was due to the incompatibility of DB1 for implementing MSCTS. The following changes were made to DB1 to make it conducive for MSCTS:

- The clock tree in DB1 is very deep and complicated. As noted in **Section 3.1**, MSCTS topologies work best if the clock tree is kept minimal. Thus, many ICGs (Power Compiler) were removed to minimize the level count and simplify the clock structure. Moreover, it is necessary to tie as many endpoints to the mesh network for achieving higher performance. Hence, the clock tree was rewired to achieve this goal.
- Timing constraints were modified since the clock structure was rewired for simplicity. However, care was taken to minimize the impact of constraint modifications on the final timing QoR.
- The floorplan of DB1 was modified to make the design mesh-friendly. The changes in floorplan were accompanied by changes in placement (**Section 3.4**).

Table 1 tabulates the results of the evaluation and compares the four clock methodologies on the basis of multiple key performance parameters. The data presented below is obtained from the 'clock_opt' database for the worst corner.

Table 1. Performance Comparisons between Clock Methodologies

Parameters	CTS (No CCD) [M-2016.12-SP5]	CTS (With CCD) [M-2016.12-SP5]	RMSCTS [M-2016.12-SP5]	SMSCTS [M-2016.12-SP5]
Clock Frequency	300 MHz	300 MHz	300 MHz	300 MHz
Number of Sinks	16,410	16,410	14,391	14,391
Clock Latency	1.290	1.704	1.382	0.660
Local Skew	0.101 (Setup),	0.421 (Setup),	0.110 (Setup),	0.011 (Setup),

	0.103 (Hold)	0.507 (Hold)	0.111 (Hold)	0.011 (Hold)
Global Skew	0.107	0.639	0.111	0.012
Buffer Count	1116	1318	578	275
Clock Depth	33	29	12	5
Datapath WNS (Setup)	-0.024	-0.013	-0.025	-0.013
Datapath TNS (Setup)	-0.182	-0.200	-1.663	-0.575
Datapath WNS (Hold)	-0.365	-0.220	-0.428	-0.2881
Datapath TNS (Hold)	-4.603	-7.019	-21.0798	-3.346
Runtime (Hour)	4.98	7.55	6.41	17.52

4.2 Summary

The results are summarized below:

- **Clock QoR:**
 - **Buffer Count:**
 - The buffer count is higher in RMSCTS relative to SMSCTS.
 - Buffer count distribution in SMSCTS:
 - 256 Mesh Drivers.
 - 19 GCT (Global Clock Tree) Drivers.
 - Buffer count distribution in RMSCTS:
 - 64 Mesh Drivers.
 - 48 Tap Drivers.
 - 10 GCT (Global Clock Tree) Drivers.
 - 456 CTS cells inserted by ICC2.
 - The impact of buffer count on the total power should be analyzed carefully. Though a reduction in buffer count tends to imply a reduction in power and area, the total power consumption is also a function of the mesh density and the drive strength of the mesh drivers.
 - CCD has a higher buffer count relative to CTS due to the additional number of cells for clock skewing.
 - Since buffer insertion is not allowed in SMSCTS, the number of clock levels tends to be smaller than all other methodologies. This explains the low clock latency observed in SMSCTS. Furthermore, it is to be noted that there is a greater control over clock levels in SMSCTS compared to all other methodologies.

- MSCTS methodologies make it possible to achieve a very low skew (Global/Local) relative to CTS and CCD. Of the four methodologies presented, **CCD has the worst skew**. However, this is expected since CCD intentionally boosts the skew for improving the timing QoR.
- RMSCTS has a higher utilization (0.5518) than SMSCTS (0.5436) due to the large number of buffers added by ICC2. The increase in buffer count results in higher routing resources.
- Since the clock structure was rewired to reduce complexity, CTS/CCD and MSCTS were implemented on two different clock domains. This is responsible for the differences in the sink count.
- **Timing QoR:**
 - Without fine-tuning, SMSCTS offers better performance than RMSCTS.
 - The timing cannot be taken as final since the design is not signal routed. Furthermore, the timing QoR depends not just on the clock network but also on the data path.

5. Future Work

- The compatibility of MSCTS with CCD.
- The implementation of MSCTS on a design with deep clock trees (ICGs).
- Understanding the impact of stuck-at-faults on the functionality of a MSCTS clock network.
- Power and OCV Analysis.
- Impact of fishbone routing on the final QoR.

6. Conclusions

The MSCTS flow has been successfully implemented on an industrial design using ICC2. The clock-design flow is evaluated and compared against the more commonly used methodologies like CTS and CCD. The challenges faced have been highlighted and their solutions/work-arounds are detailed. The results demonstrate the engineering trade-offs involved in the flow and offers valuable insights to those intending to incorporate MSCTS in their designs.

7. Acknowledgements

I would like to thank my manager, **Mr. Boon Loong Yong** for his support and valuable opinions. I would also like to extend my thanks to **Mr. Nicholas Yee Wee Han** and the support team at Synopsys for their promptness and excellent feedback.

8. References

- [1] Synopsys Documentation.
- [2] SolvNet